



Geometric Algorithms

Module - 6

BCSE204L - Design and Analysis of Algorithms



Geometric Algorithms

- In modern engineering and mathematics, computational geometry has applications in, among other fields, computer graphics, robotics, VLSI design, computer-aided design, and statistics.
 - The input to a computational-geometry problem is typically a description of a set of geometric objects, such as **a set of points, a set of line segments, or the vertices of a polygon** in counterclockwise order.
 - The output is often a response to a query about the objects, such as whether any of the lines intersect, or perhaps a new geometric object, such as the convex hull (smallest enclosing convex polygon) of the set of points.
- 

Geometric Algorithms

- A few computational-geometry algorithms in two dimensions, that is, in the plane.
- Each input object is represented as a set of points $\{p_i\}$, where each $p_i = (x_i, y_i)$ and $x_i, y_i \in R$.
- For example, an n -vertex polygon P is represented by a sequence $p_0, p_1, p_2, \dots, p_{n-1}$ of its vertices in order of their appearance on the boundary of P .
- Computational geometry can also be performed in three dimensions, and even in higher-dimensional spaces, but such problems and their solutions can be very difficult to visualize.
- Even in two dimensions, however, we can see a good sample of computational-geometry techniques.

Geometric Algorithms – Line Segment

- Properties of line segments:
- A convex combination of two distinct points

$p_1 = (x_1, y_1)$ and $p_2 = (x_2, y_2)$
is any point $p_3 = (x_3, y_3)$ such that for some in the range $0 \leq \alpha \leq 1$,
we have $x_3 = \alpha x_1 + (1 - \alpha)x_2$ and $y_3 = \alpha y_1 + (1 - \alpha)y_2$.

We also write that $p_3 = \alpha p_1 + (1 - \alpha)p_2$.

Geometric Algorithms – Line Segment

- **Properties of line segments:**
- Intuitively, p_3 is any point that is on the line passing through p_1 and p_2 and is on or between p_1 and p_2 on the line.
- Given two distinct points p_1 and p_2 , the **line segment** $\overline{p_1 p_2}$ is the set of convex combinations of p_1 and p_2 .
- We call p_1 and p_2 the **endpoints of segment** $\overline{p_1 p_2}$.
- Sometimes the ordering of p_1 and p_2 matters, and we speak of the **directed segment** $\overrightarrow{p_1 p_2}$.
- If p_1 is the origin $(0, 0)$, then we can treat the directed segment as the **vector** p_2 .

Geometric Algorithms – Line Segment

- **Explore few questions:**

- Given two directed segments $\overrightarrow{p_1p_2}$ and $\overrightarrow{p_1p_3}$, is $\overrightarrow{p_1p_2}$ clockwise from $\overrightarrow{p_1p_3}$ with respect to their common endpoint p_1 .
- Given two line segments $\overline{p_1p_2}$ and $\overline{p_2p_3}$, if we traverse $\overline{p_1p_2}$ and then $\overline{p_2p_3}$, do we make a left turn at point p_2 .
- Do line segments $\overline{p_1p_2}$ and $\overline{p_3p_4}$ intersect?

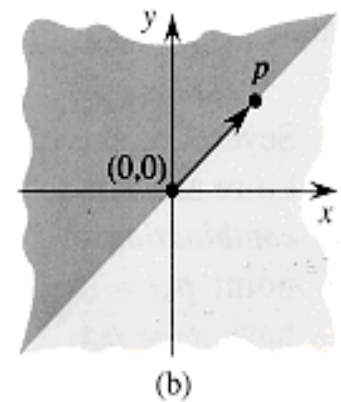
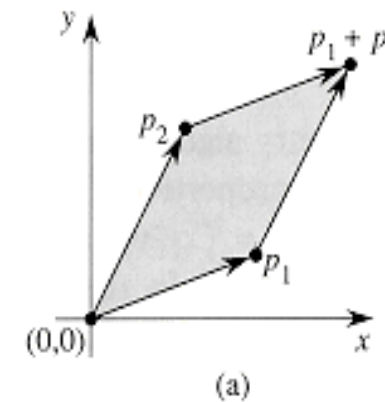
Geometric Algorithms – Line Segment

- There are no restriction on the given points.
- We can answer each question in $O(1)$ time.
- We will use only additions, subtractions, multiplications and comparisons.
- We neither need division or trigonometric functions, which are expensive and prone to round off error.
- To determine whether two segments intersect, we compute the line equation of the form $y = mx + b$ for each segment,

Geometric Algorithms – Line Segment

- **Cross Products**

- Computing cross products $p_1 \times p_2$ can be interpreted as the signed area of parallelogram formed by the points $(0,0)$, p_1 , p_2 , and $p_1 + p_2 = (x_1 + x_2, y_1 + y_2)$.
- Definition of the cross product as determinant of a matrix



- $p_1 \times p_2 = \det \begin{pmatrix} x_1 & x_2 \\ y_1 & y_2 \end{pmatrix} = x_1 y_2 - x_2 y_1 = -p_2 \times p_1$

Geometric Algorithms – Line Segment

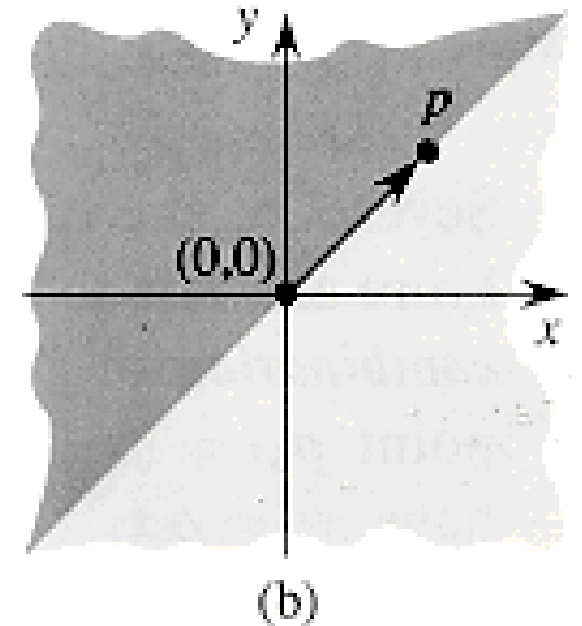
- **Cross Products**

- The cross product is a three-dimensional concept.
- It is a vector that is perpendicular to both p_1 and p_2 according to the "right-hand rule" and whose magnitude is $|x_1y_2 - x_2y_1|$.
- The convenient to treat the cross product simply as the value $x_1y_2 - x_2y_1$.

Geometric Algorithms – Line Segment

- **Cross Products**

- If $p_1 \times p_2$ is **positive**, then p_1 is **clockwise** from p_2 with respect to the origin $(0,0)$; if this cross product is **negative**, then p_1 is **counterclockwise** from p_2 . Figure shows the clockwise and counterclockwise regions relative to a vector p .
- A boundary condition arises if the **cross product is zero**; in this case, the vectors are **collinear**, pointing in either the same or opposite directions.



Geometric Algorithms – Line Segment

- **Cross Products**

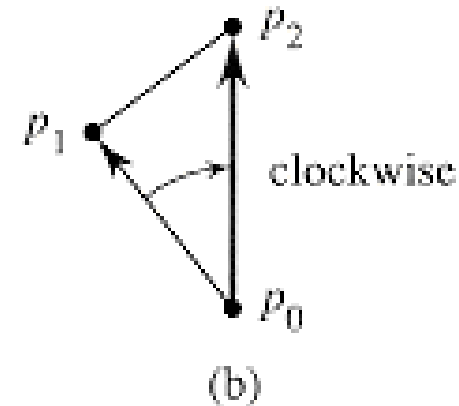
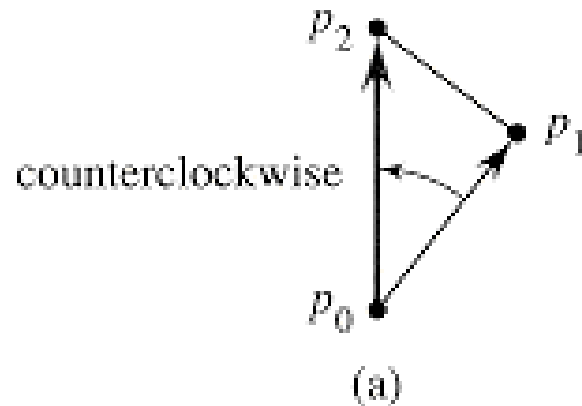
- To determine whether a directed segment $\overrightarrow{p_0 p_1}$ is clockwise from a directed segment $\overrightarrow{p_0 p_2}$ with respect to their common endpoint p_0 , we simply translate to use p_0 as the origin.
- That is, we let $p_1 - p_0$ denote the vector $p'_1 = (x'_1, y'_1)$, where $x'_1 = x_1 - x_0$ and $y'_1 = y_1 - y_0$, and we define $p_2 - p_0$ similarly.
- We then compute the cross product
$$(p_1 - p_0) \times (p_2 - p_0) = (x_1 - x_0)(y_2 - y_0) - (x_2 - x_0)(y_1 - y_0).$$
- If this cross product is **positive, then is clockwise** from ; if **negative, it is counterclockwise**.

Geometric Algorithms – Line Segment

- **Determine Whether Consecutive segments turn left or right**
- Given two consecutive line segments $\overline{p_0p_1}$ and $\overline{p_1p_2}$ to check whether it turn left or right to point p_1 .
- Cross product allow us to find turn left or right without computing the angle.
- We compute $(p_2 - p_0) \times (p_1 - p_0)$
- If the sign of this cross product **is negative** then $\overline{p_0p_2}$ is **counter clockwise** w.r.t $\overline{p_0p_1}$, thus a **left turn** at p_1 .
- If the sign of this cross product **is positive** then $\overline{p_0p_2}$ is **clockwise** w.r.t $\overline{p_0p_1}$, thus a **right turn** at p_1 .

Geometric Algorithms – Line Segment

- **Determine Whether Consecutive segments turn left or right**
- Figure a for cross product **is negative** then $\overline{p_0p_2}$ is **counter clockwise** w.r.t $\overline{p_0p_1}$, thus a **left turn** at p_1 .
- Figure b for cross product **is positive** then $\overline{p_0p_2}$ is **clockwise** w.r.t $\overline{p_0p_1}$, thus a **right turn** at p_1 .
-





Geometric Algorithms – Line Segment

- **Determine Whether Two Line segments turn Intersect**
 - Two methods
 - Quick Rejection and Straddles
 - Cross product
- 

Geometric Algorithms – Line Segment

- **Two Line - Intersect - Quick Reject and Straddles**

- First stage: **Quick Rejection** – line segment cannot intersect if their **Bounding Box** do not intersect.
- **Bounding Box** – smallest rectangle that contains the figure and whose segments are parallel to the x-axis and y-axis.
- The bounding box of line segment $\overline{p_1p_2}$ is represented by rectangle $(\widehat{p_1}, \widehat{p_2})$ with lower left point $\widehat{p_1} = (\widehat{x_1}, \widehat{y_1})$ and upper right point $\widehat{p_2} = (\widehat{x_2}, \widehat{y_2})$, where

$$\widehat{x_1} = \min(x_1, x_2), \widehat{y_1} = \min(y_1, y_2), \widehat{x_2},$$

$$\widehat{x_2} = \max(x_1, x_2), \widehat{y_2} = \max(y_1, y_2)$$

- where $(\widehat{p_1}, \widehat{p_2})$ and $(\widehat{p_3}, \widehat{p_4})$.

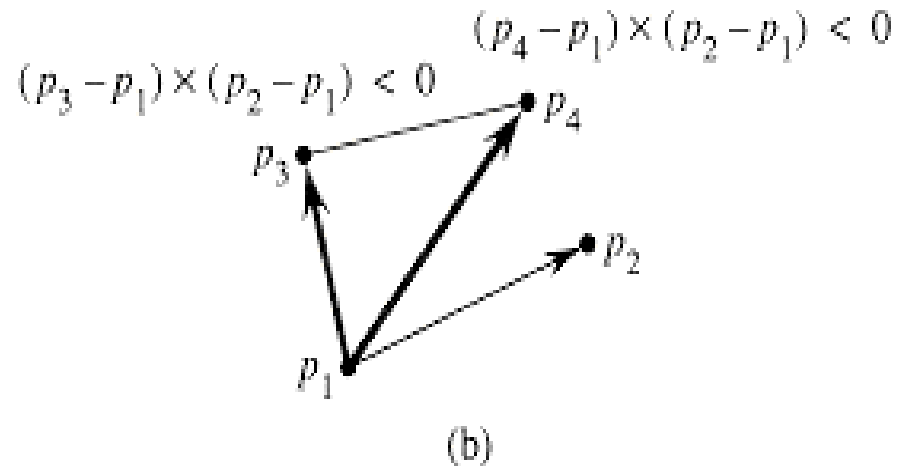
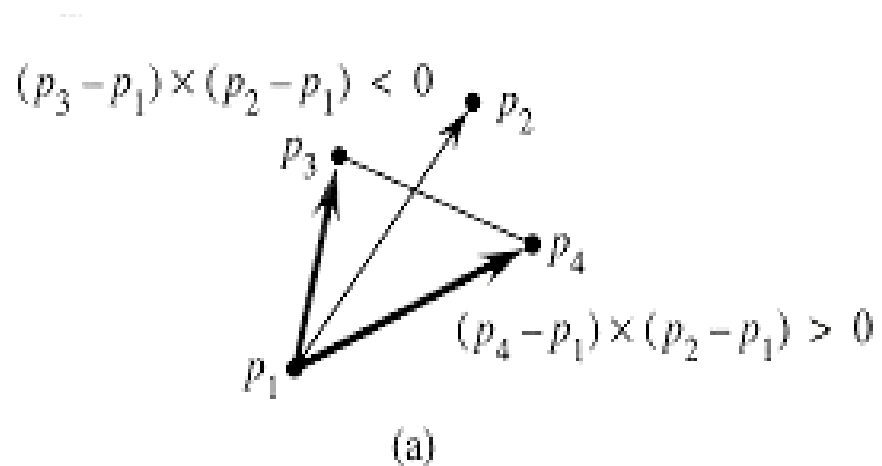
Geometric Algorithms – Line Segment

- **Two Line - Intersect - Quick Reject and Straddles**

- Two rectangles, represented by lower left $(\widehat{p}_1, \widehat{p}_2)$ and upper right $(\widehat{p}_3, \widehat{p}_4)$ points, intersect iff the conjunction
- $(\widehat{x}_2 \geq \widehat{x}_3) \wedge (\widehat{x}_4 \geq \widehat{x}_1) \wedge (\widehat{y}_2 \geq \widehat{y}_3) \wedge (\widehat{y}_4 \geq \widehat{y}_1)$ is true.
- The first two comparison determine whether rectangle intersect in x and other two comparison determine whether the rectangle intersect in y.
- The second stage in determining whether two line segments intersect decides whether each segment “**straddles**” the line containing the other.
- A segment $\overline{p_1 p_2}$ **straddles** a line if point p_1 lies on one side of the line and p_2 lies on other side.
- If p_1 or p_2 lies on the line, then we say that the segment straddles the line.

Geometric Algorithms – Line Segment

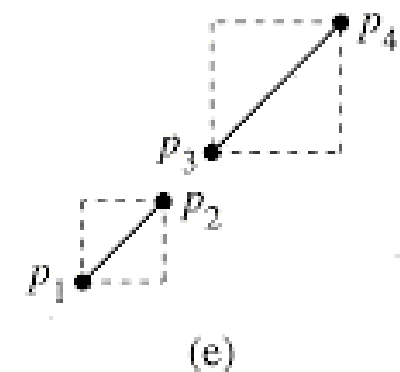
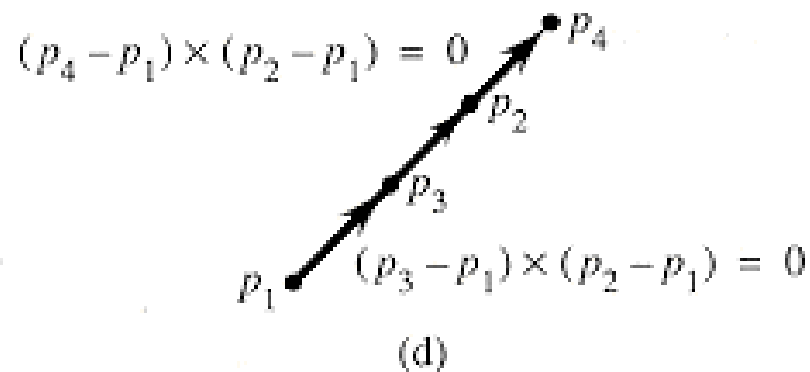
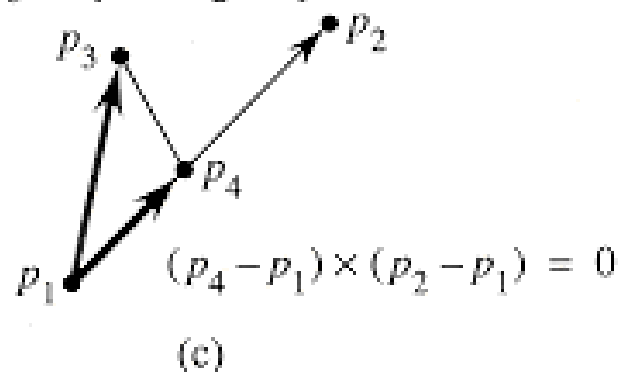
- **Line segment $\overline{p_3p_4}$ straddles the line segment $\overline{p_1p_2}$**
- (a) If it does straddle, then the signs of the cross products $(p_3 - p_1) \times (p_2 - p_1)$ and $(p_4 - p_1) \times (p_2 - p_1)$ differ.
- (b) If it does not straddle, then the signs of the cross products are the same.



Geometric Algorithms – Line Segment

- Line segment $\overline{p_3p_4}$ straddles the line segment $\overline{p_1p_2}$
- (c)-(d) Boundary cases in which at least one of the cross products is zero and the segment straddles.
- (e) A boundary case in which the segments are collinear but do not intersect. Both cross products are zero, but they would not be computed by our algorithm because the segments fail the quick rejection test--their bounding boxes do not intersect.

$$(p_3 - p_1) \times (p_2 - p_1) < 0$$



Geometric Algorithms – Line Segment

- **Determine Whether Two Line segments turn Intersect using Cross Product**
- Given two line segments $\overline{p_3p_4}$ straddles the line segment $\overline{p_1p_2}$
- The cross product $(p_3 - p_1) \times (p_2 - p_1)$ and $(p_4 - p_1) \times (p_2 - p_1)$ are different.
- A boundary condition occurs if either cross product is zero
- Either p_3 or p_4 lies on the line containing segment $\overline{p_1p_2}$.
- A final boundary condition occurs if one or both of the segments has zero length, that is endpoints are coincident.
- If one segment, say $\overline{p_3p_4}$, has zero length, then the segments intersect if and only if the cross product $(p_3 - p_1) \times (p_2 - p_1)$ is zero.

Geometric Algorithms – Line Segment

- **Determine Whether any pair of segments Intersect**
- Sweeping : an algorithm for determining whether any two line segments in a set of segments intersect.
- The algorithm runs in $O(n \lg n)$ time, where n is the number of segments we are given. It determines only whether or not any intersection exists; it does not print all the intersections.

Geometric Algorithms – Line Segment

- **Sweeping**

- In **sweeping**, an imaginary vertical **sweep line** passes through the given set of geometric objects, usually from left to right.
- The spatial dimension that the sweep line moves across, in this case the x-dimension, is treated as a dimension of time.
- Sweeping provides a method for ordering geometric objects, usually by placing them into a dynamic data structure, and for taking advantage of relationships among them.
- The line-segment-intersection algorithm in this section considers all the line-segment endpoints in left-to-right order and checks for an intersection each time it encounters an endpoint.

Geometric Algorithms – Line Segment

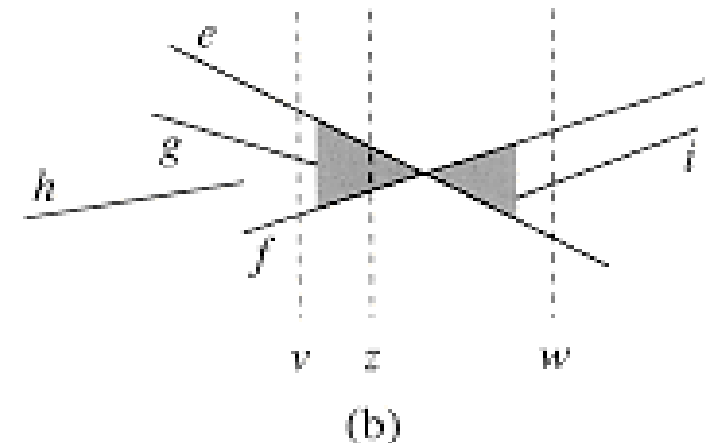
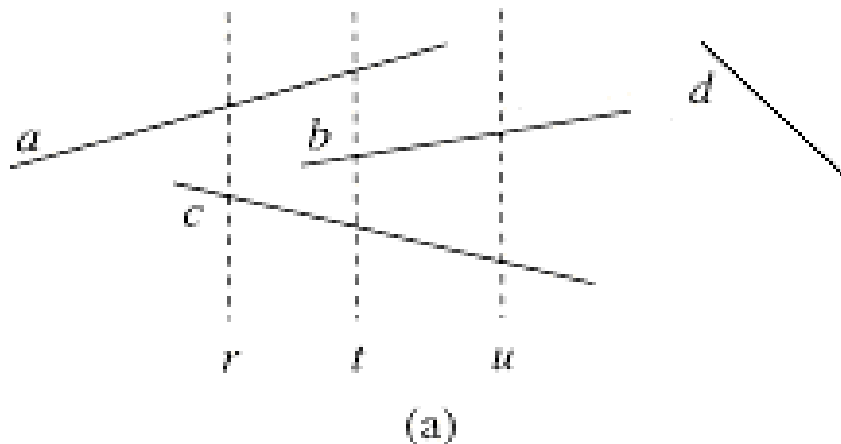
- **Sweeping**

- Algorithm for determining whether any two of n line segments intersect makes two simplifying assumptions.
- First, we assume that no input segment is vertical.
- Second, we assume that no three input segments intersect at a single point.
- Indeed, removing such simplifying assumptions and dealing with boundary conditions is often the most difficult part of programming computational-geometry algorithms and proving their correctness.

Geometric Algorithms – Line Segment

- **Sweeping**

- The ordering among line segments at various vertical sweep lines. (a) We have $a >_r c, a >_t b, b >_t c, a >_t c$, and $b >_u c$. Segment d is comparable with no other segment shown. (b) When segments e and f intersect, their orders are reversed: we have $e >_v f$ but $f >_w e$. Any sweep line (such as z) that passes through the shaded region has e and f consecutive in its total order.



Geometric Algorithms – Line Segment

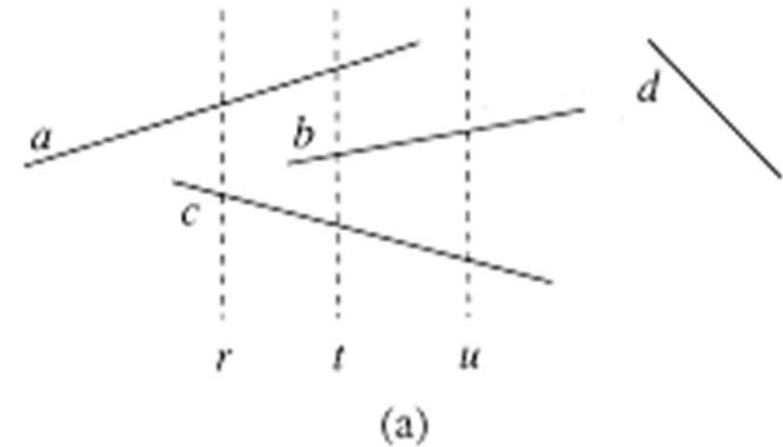
- **Ordering Segments**

- Assume that there are no vertical segments, any input segment that intersects a given vertical sweep line intersects it at a single point.
- We can thus order the segments that intersect a vertical sweep line according to the y -coordinates of the points of intersection.
- Two nonintersecting segments s_1 and s_2 We say that these segments are comparable at x if the vertical sweep line with x -coordinate x intersects both of them.
- We say that s_1 is above s_2 at x , written $s_1 >_x s_2$, if s_1 and s_2 are comparable at x and the intersection of s_1 with the sweep line at x is higher than the intersection of s_2 with the same sweep line.

Geometric Algorithms – Line Segment

- **Ordering Segments**

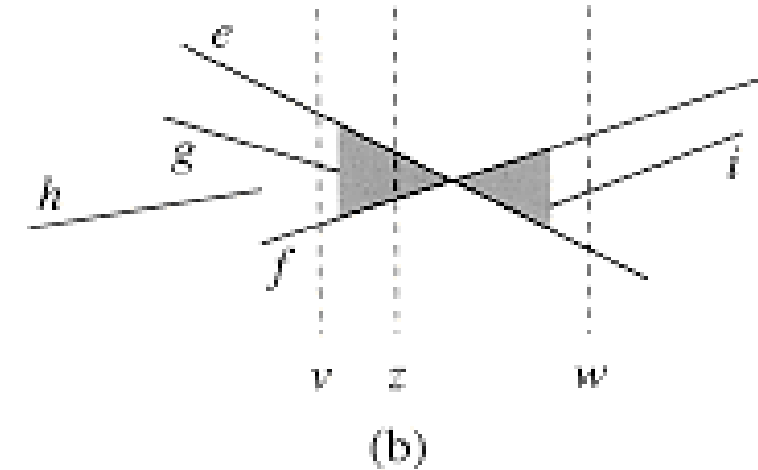
- For example, we have the relationships $a >_r c, a >_t b, b >_t c, a >_t c$, and $b >_u c$. Segment d is not comparable with any other segment.
- For any given x , the relation " $>_x$ " is a total order on segments that intersect the sweep line at X .
- The order may differ for differing values of x , however, as segments enter and leave the ordering.
- A segment enters the ordering when its left endpoint is encountered by the sweep, and it leaves the ordering when its right endpoint is encountered.



Geometric Algorithms – Line Segment

- **Ordering Segments**

- As Figure (b) shows, their positions in the total order are reversed.
- Sweep lines v and w are to the left and right, respectively, of the point of intersection of segments e and f , and we have $e >_v f$ and $f >_w e$.
- Note that because we assume that no three segments intersect at the same point, there must be some vertical sweep line x for which intersecting segments e and f are *consecutive* in the total order $>_x$.
- Any sweep line that passes through the shaded region of Figure (b), such as z , has e and f consecutive in its total order.



Geometric Algorithms – Line Segment

- **Moving the Sweeping Line**
- Sweeping algorithms typically manage two sets of data:
 - 1. The **sweep-line status** gives the relationships among the objects intersected by the sweep line.
 - 2. The **event-point schedule** is a sequence of x -coordinates, ordered from left to right, that defines the halting positions of the sweep line. We call each such halting position an **event point**. Changes to the sweep-line status occur only at event points.

Geometric Algorithms – Line Segment

- **Moving the Sweeping Line**

- For some algorithms, the event-point schedule is determined dynamically as the algorithm progresses.
- The algorithm at hand, however, determines the event points statically, based solely on simple properties of the input data. In particular, each segment endpoint is an event point.
- We sort the segment endpoints by increasing x-coordinate and proceed from left to right.
- We insert a segment into the sweep-line status when its left endpoint is encountered, and we delete it from the sweep-line status when its right endpoint is encountered. Whenever two segments first become consecutive in the total order, we check whether they intersect.

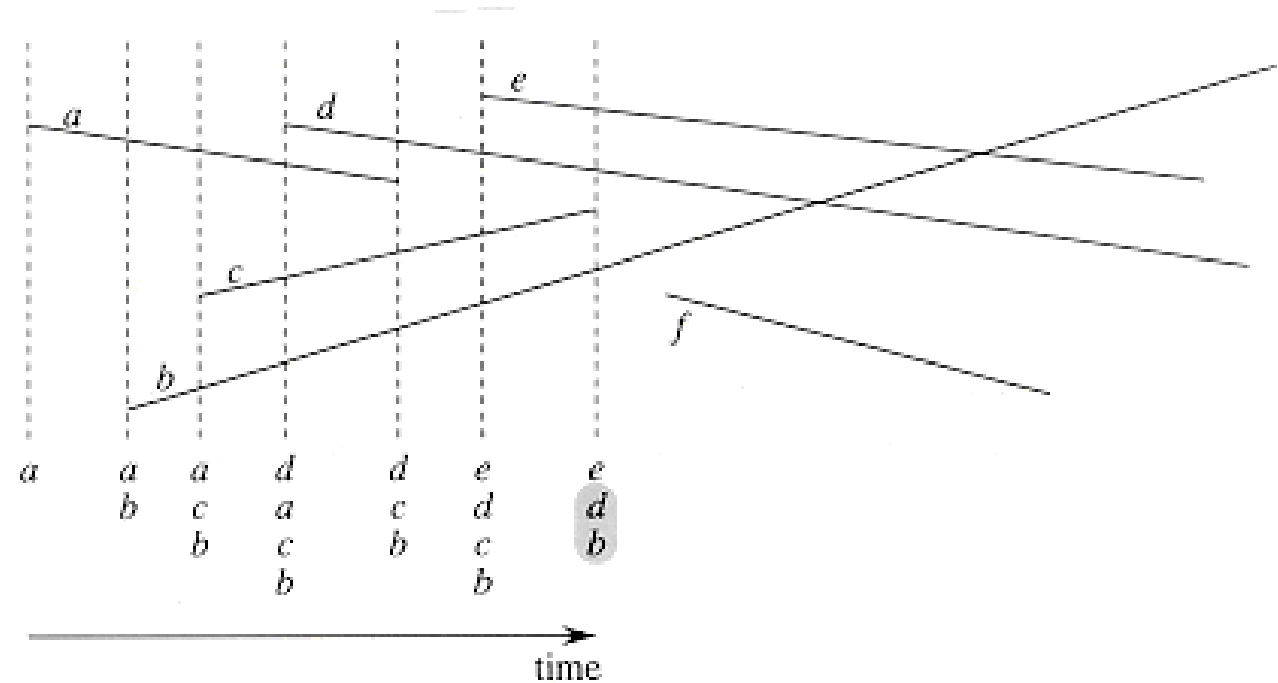
Geometric Algorithms – Line Segment

- **Moving the Sweeping Line**

- The sweep-line status is a total order T , for which we require the following operations:
- $\text{INSERT}(T, s)$: insert segment s into T .
- $\text{DELETE}(T, s)$: delete segment s from T .
- $\text{ABOVE}(T, s)$: return the segment immediately above segment s in T .
- $\text{BELOW}(T, s)$: return the segment immediately below segment s in T .
- If there are n segments in the input, we can perform each of the above operations in $O(\lg n)$ time using red-black trees.

Intersection

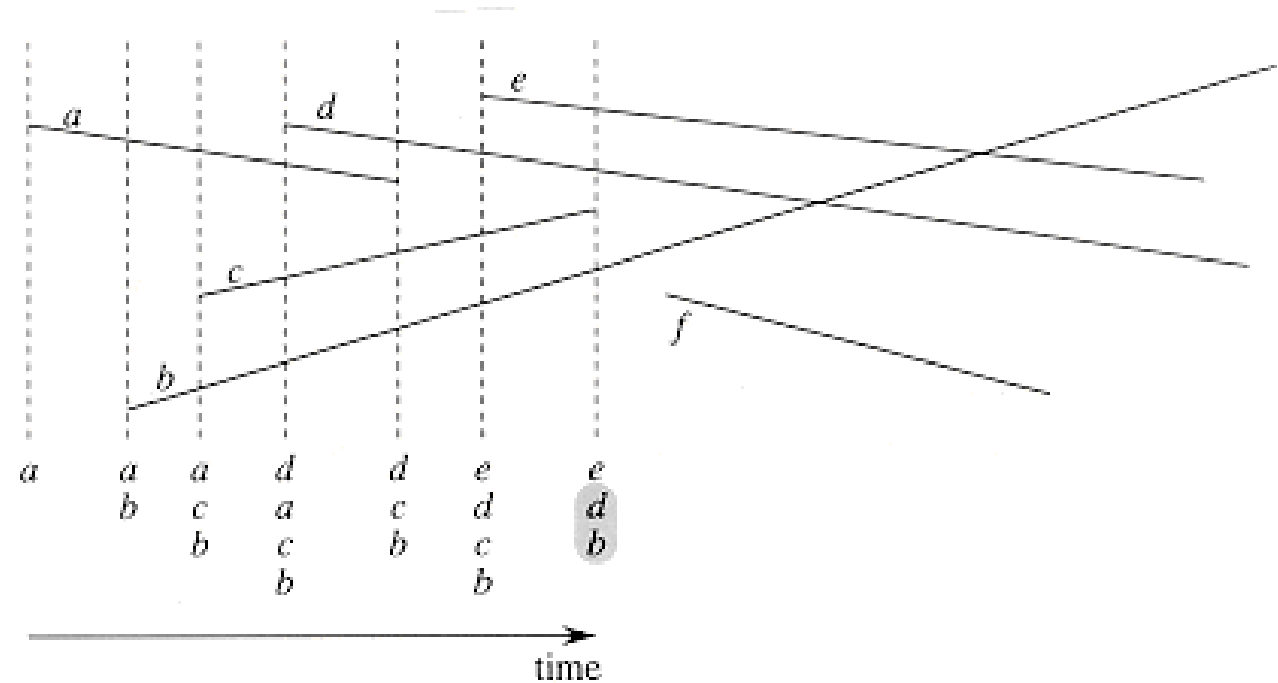
- The following algorithm takes as input a set S of n line segments, returning the boolean value TRUE if any pair of segments in S intersects, and FALSE otherwise.



Geometric Algorithms – Line Segment

• Segment Intersection Pseudocode

- The execution of ANY-SEGMENTS-INTERSECT. Each dashed line is the sweep line at an event point, and the ordering of segment names below each sweep line is the total order T at the end of the for loop in which the corresponding event point is processed. The intersection of segments d and b is found when segment c is deleted.



Geometric Algorithms – Line Segment

ANY-SEGMENTS-INTERSECT(S)

$T \leftarrow \emptyset$

Sort the endpoints of the segments in S from left to right, breaking ties by putting points with lower y -coordinates first

FOR each point p in the sorted list of endpoints **DO**

IF p is the left endpoint of a segment s **THEN**

$\text{INSERT}(T, s)$

IF $(\text{ABOVE}(T, s)$ exists and

$\text{intersects } s)$ or $(\text{BELOW}(T, s)$ exists and intersects $s)$

THEN return TRUE

IF p is the right endpoint of a segment s **THEN**

IF both $\text{ABOVE}(T, s)$ and $\text{BELOW}(T, s)$ exist and $\text{ABOVE}(T, s)$ intersects $\text{BELOW}(T, s)$ **THEN**

RETURN TRUE

$\text{DELETE}(T, s)$

RETURN FALSE

Geometric Algorithms – Line Segment

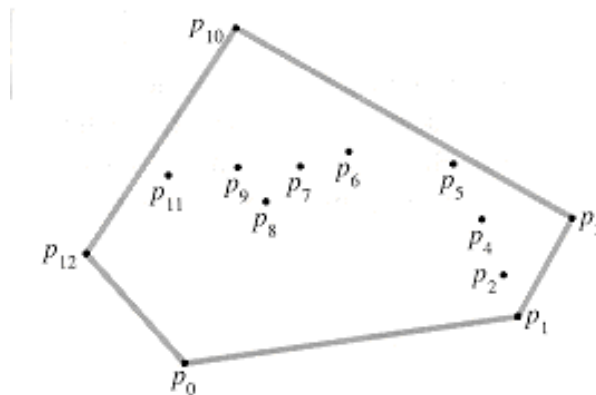
- **Running Time**

- If there are n segments in set S , then ANY-SEGMENTS-INTERSECT runs in time $O(n \lg n)$.
- Line 1 takes $O(1)$ time.
- Line 2 takes $O(n \lg n)$ time, using merge sort or heapsort.
- Since there are $2n$ event points, the for loop of lines 3-11 iterates at most $2n$ times.
- Each iteration takes $O(\lg n)$ time, total time is thus **$O(n \lg n)$** .

Geometric Algorithms – Convex Hull

- **Convex Hull**

- The convex hull of a set Q of points is the smallest convex polygon P for which each point in Q is either on the boundary of P or in its interior.
- We denote the convex hull of Q by $CH(Q)$. Intuitively, we can think of each point in Q as being a nail sticking out from a board.
- The convex hull is then the shape formed by a tight rubber band that surrounds all the nails. Figure shows a set of points and its convex hull.



Geometric Algorithms – Convex Hull

- **Convex Hull Algorithm**

- Two algorithms output the vertices of the convex hull in counterclockwise order.
- The first, known as **Graham's scan**, runs in $O(n \lg n)$ time.
- The second, called **Jarvis's march**, runs in $O(nh)$ time, where h is the number of vertices of the convex hull.
- As can be seen from Figure, every vertex of $CH(Q)$ is a point in Q . Both algorithms exploit this property, deciding which vertices in Q to keep as vertices of the convex hull and which vertices in Q to throw out.



Geometric Algorithms – Convex Hull

- **Convex Hull**
- Graham's scan and Jarvis's march use a technique called "**rotational sweep**," processing vertices in the order of the polar angles they form with a reference vertex.

Convex Hull – Graham's Scan

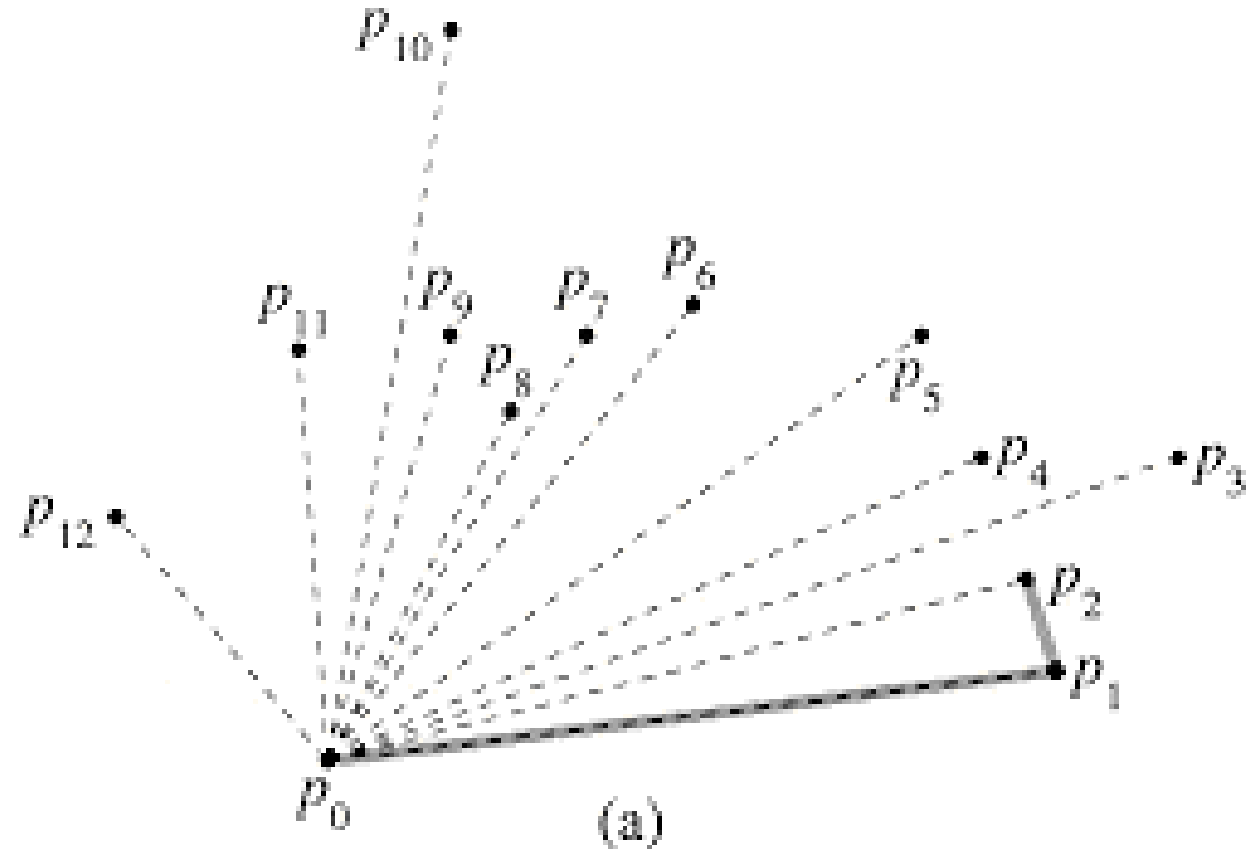
- **Graham's Scan**

- Graham's scan solves the convex-hull problem by maintaining a stack S of candidate points.
- Each point of the input set Q is pushed once onto the stack, and the points that are not vertices of $CH(Q)$ are eventually popped from the stack.
- When the algorithm terminates, stack S contains exactly the vertices of $CH(Q)$, in counterclockwise order of their appearance on the boundary.

Convex Hull – Graham's Scan

- **Graham's Scan**

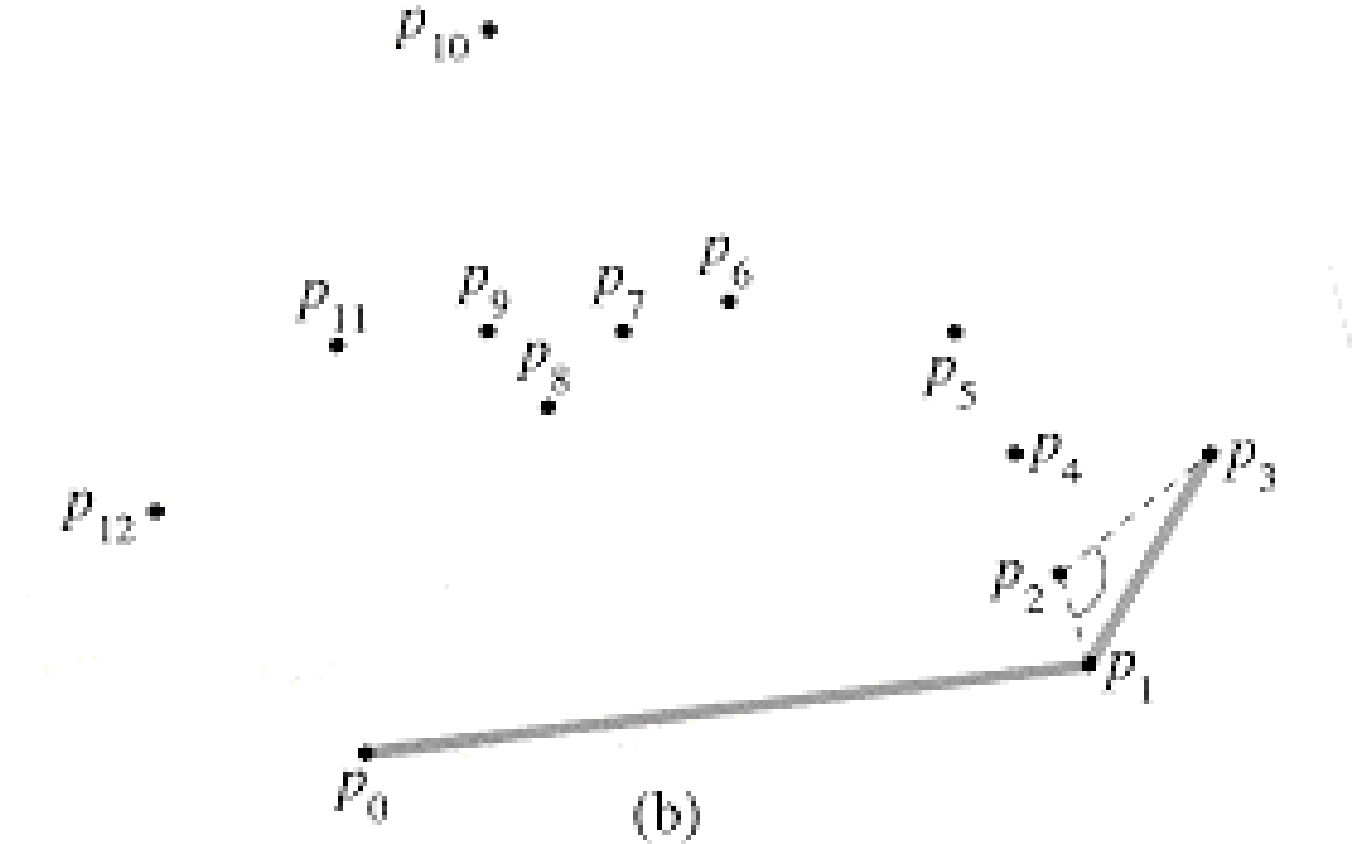
- The execution of **GRAHAM-SCAN** on the set Q of Figure. The current convex hull contained in stack S is shown in gray at each step. (a) The ordered polar angles of $p_1, p_2, \dots, p_{12}$ relative to p_0 and the initial stack S containing p_0, p_1 , and p_2 .



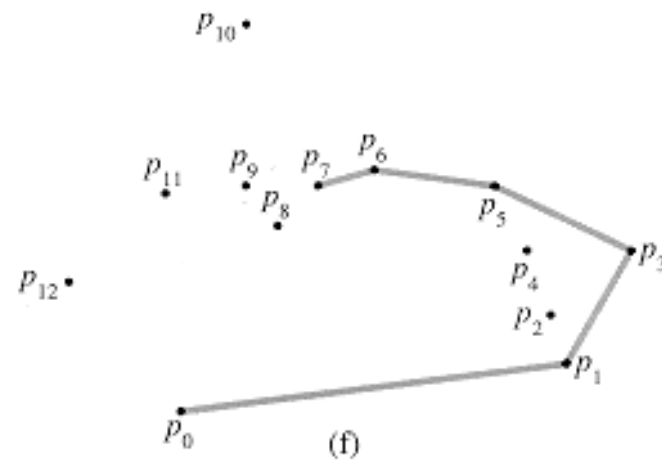
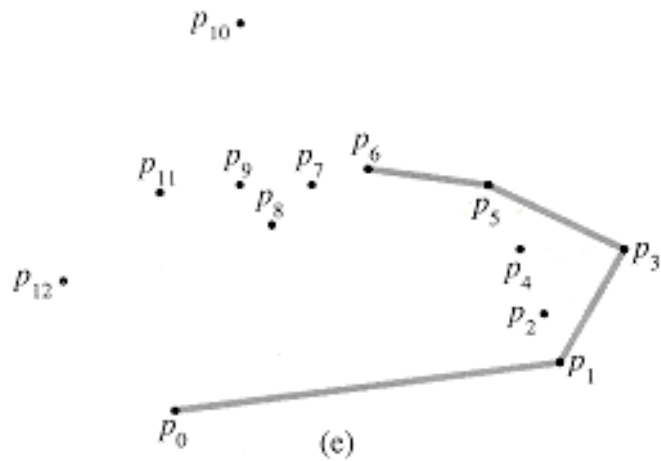
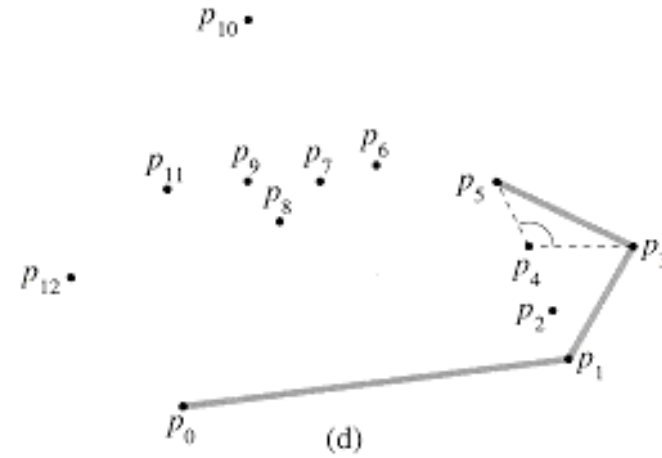
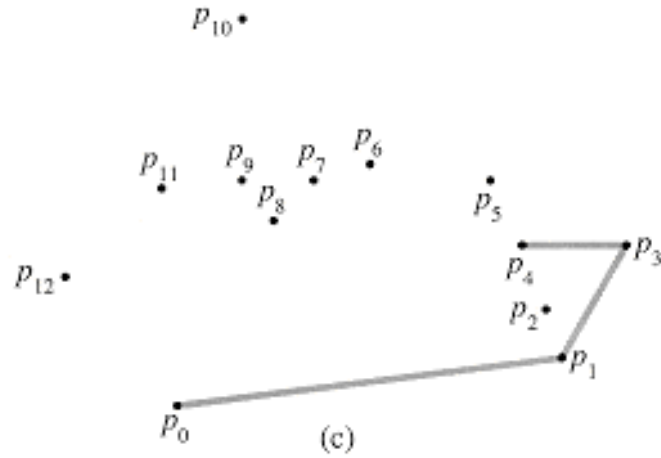
Convex Hull – Graham's Scan

- **Graham's Scan**

- (b)-(k) Stack S after each iteration of the for loop of lines 7-10. Dashed lines show nonleft turns, which cause points to be popped from the stack.



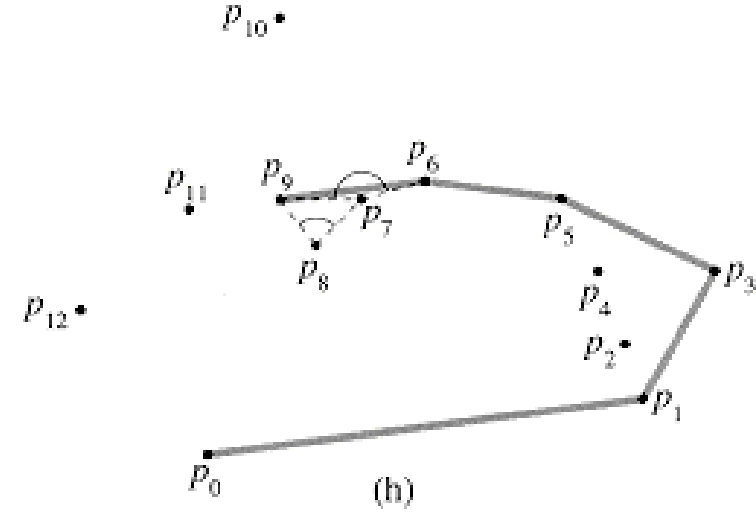
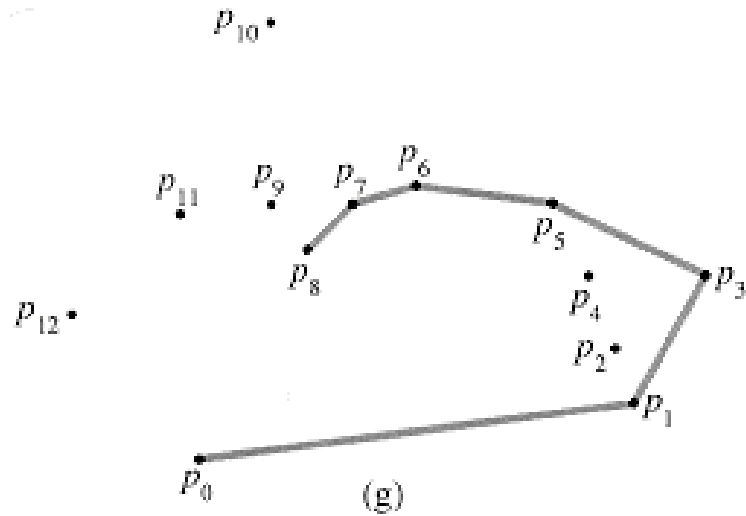
Convex Hull – Graham's Scan



Convex Hull – Graham's Scan

- **Graham's Scan**

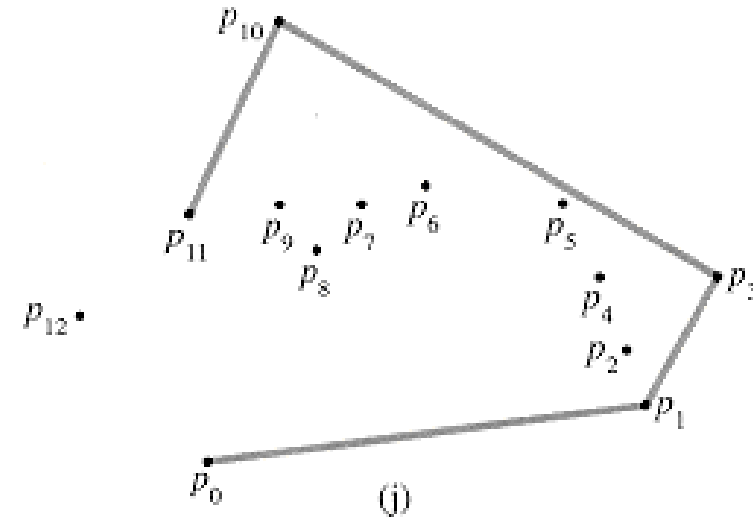
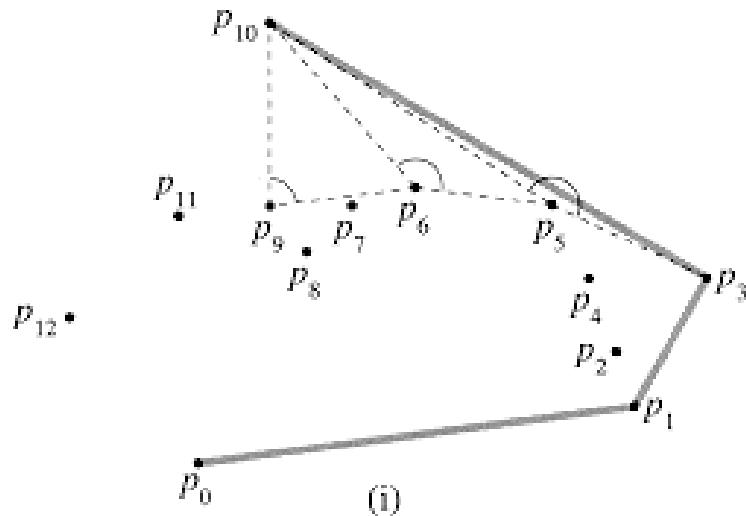
- (b)-(k) Stack S after each iteration of the for loop of lines 7-10. Dashed lines show nonleft turns, which cause points to be popped from the stack.



Convex Hull – Graham's Scan

- **Graham's Scan**

- In part (h), for example, the right turn at angle $p_7p_8p_9$ causes p_8 to be popped, and then the right turn at angle $p_6p_7p_9$ causes p_7 to be popped. (I) The convex hull returned by the procedure, which matches that of previous Figure



Convex Hull – Graham's Scan

GRAHAM-SCAN(Q)

Let p_0 be the point in Q with the minimum y-coordinate, or the leftmost such point in case of a tie

Let $p_1, p_2, \dots, p_m$ be the remaining points in Q , sorted by polar angle in counterclockwise order around p_0 (if more than point has the same angle, remove all but the one that is farthest from p_0)

$\text{top}[S] \leftarrow 0$, PUSH(p_0, S), PUSH(p_1, S), PUSH(p_2, S)

FOR i 3 to m **DO**

WHILE the angle formed by points NEXT-TO-TOP(S), TOP(S), and p_i makes a nonleft turn **DO**

 POP(S)

 PUSH(S, p_i)

RETURN S



Convex Hull – Graham's Scan

- **Graham's Scan**
- the running time of GRAHAM-SCAN is $O(n \lg n)$, where $n = |Q|$.

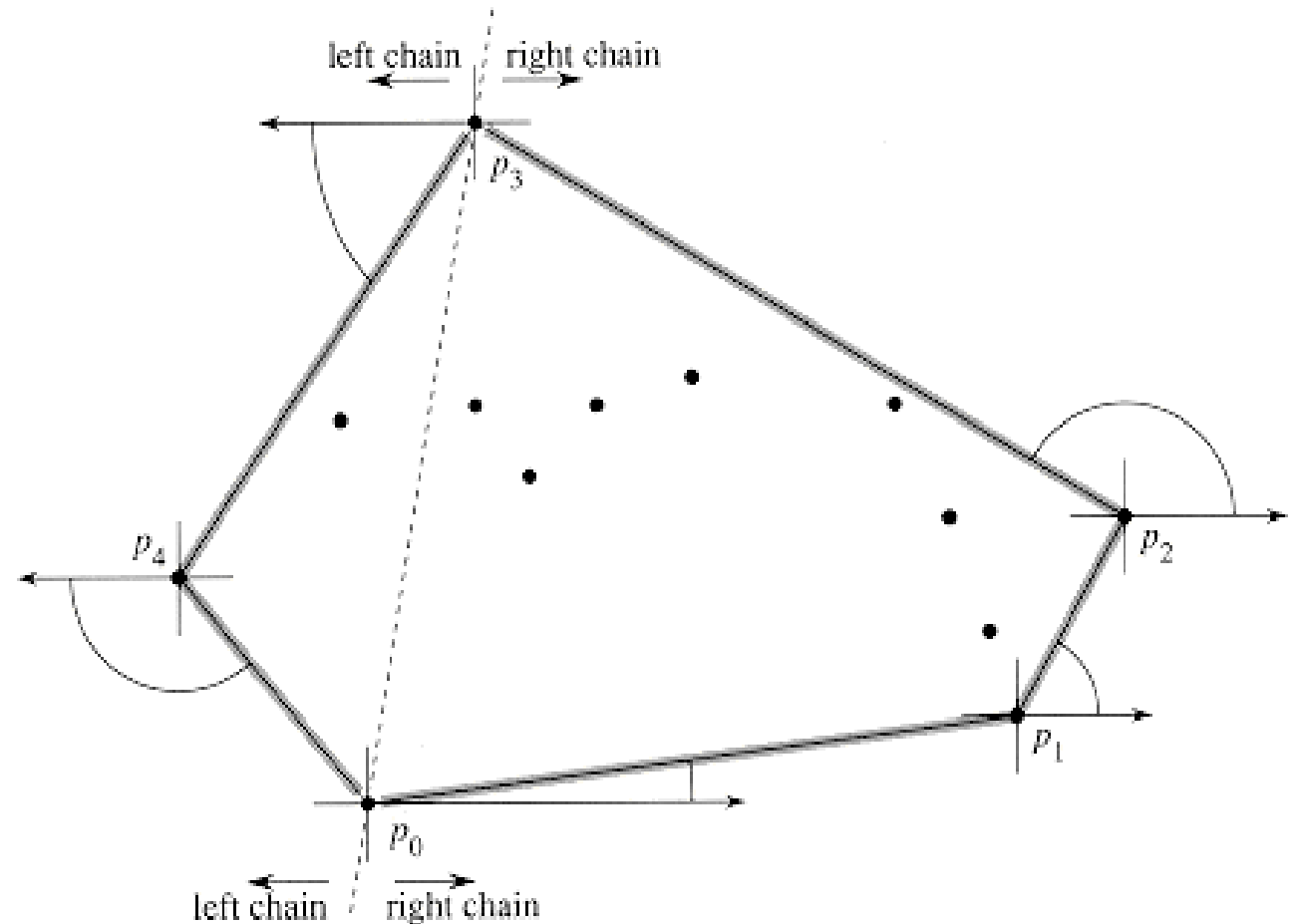
Convex Hull – Jarvis's March

- **Jarvis's March**
- **Jarvis's march** computes the convex hull of a set Q of points by a technique known as **package wrapping** (or **gift wrapping**).
- The algorithm runs in time $O(nh)$, where h is the number of vertices of $CH(Q)$.
- When h is $o(\lg n)$, Jarvis's march is asymptotically faster than Graham's scan.

Convex Hull – Jarvis's March

- **Jarvis's March**

- The operation of Jarvis's march.
- The first vertex chosen is the lowest point p_0 .
- The next vertex, p_1 , has the least polar angle of any point with respect to p_0 .
- Then, p_2 has the least polar angle with respect to p_1 . The right chain goes as high as the highest point p_3 .
- Then, the left chain is constructed by finding least polar angles with respect to the negative x-axis.



Convex Hull – Jarvis's March

- **Jarvis's March**

- Jarvis's march simulates wrapping a taut piece of paper around the set Q .
- We start by taping the end of the paper to the lowest point in the set, that is, to the same point p_0 with which we start Graham's scan.
- This point is a vertex of the convex hull.
- We pull the paper to the right to make it taut, and then we pull it higher until it touches a point.
- This point must also be a vertex of the convex hull.
- Keeping the paper taut, we continue in this way around the set of vertices until we come back to our original point p_0 .

Convex Hull – Jarvis's March

- **Jarvis's March**

- Jarvis's march builds a sequence $H = p_0, p_1, \dots, p_{h-1}$ of the vertices of $CH(Q)$. We start with p_0 .
- Next convex hull vertex p_1 has the least polar angle with respect to p_0 . (In case of ties, we choose the point farthest from p_0 .)
- Similarly, p_2 has the least polar angle with respect to p_1 , and so on.
- When we reach the highest vertex, say p_k (breaking ties by choosing the farthest such vertex), we have constructed, as Figure 35.10 shows, the right chain of $CH(Q)$.
- To construct the left chain, we start at p_k and choose p_{k+l} as the point with the least polar angle with respect to p_k , but from the negative x-axis.
- We continue on, forming the left chain by taking polar angles from the negative x-axis, until we come back to our original vertex p_0 .

Convex Hull – Jarvis's March

- **Jarvis's March**

- Jarvis's march in one conceptual sweep around the convex hull, that is, without separately constructing the right and left chains.
- Such implementations typically keep track of the angle of the last convex-hull side chosen and require the sequence of angles of hull sides to be strictly increasing (in the range of 0 to 2 radians).
- The advantage of constructing separate chains is that we need not explicitly compute angles; the techniques of Section 35.1 suffice to compare angles.

Convex Hull – Jarvis's March

- **Jarvis(S)** // S is the set of points

pointOnHull = leftmost point in S //which is guaranteed to be part of the Hull(S)

i = 0

REPEAT

P[i] = pointOnHull, endpoint = S[0] // initial endpoint for a candidate edge on the hull

FOR j from 1 to |S|

IF (endpoint == pointOnHull) or (S[j] is on left of line from P[i] to endpoint)

 endpoint = S[j] // found greater left turn, update endpoint

i = i+1

pointOnHull = endpoint

until endpoint == P[0]

Convex Hull – Jarvis's March

- **Jarvis's March**

- Jarvis's march has a running time of $O(nh)$.
- For each of the h vertices of $CH(Q)$, we find the vertex with the minimum polar angle.
- Each comparison between polar angles takes $O(1)$ time,
- We can compute the minimum of n values in $O(n)$ time if each comparison takes $O(1)$ time.
- Thus, Jarvis's march takes $O(nh)$ time.



Thank you

Module 6

BCSE204L - Design and Analysis of
Algorithm